BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 1

C PROGRAMMING

Unit 1

Introduction to C Programming

Table of Contents

SL No	Topic		Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction		-	-	3 - 4
	1.1	Overview in C	-	-	
	1.2	History and evolution of the C programming	-	-	
	1.3	Objectives	-	-	
2	Features of C and its Basic Structure		1	1, 2	5 - 10
	2.1	Programming Style:	-	-	
	2.2	Simple C Programs	-	-	
3	Summary		-	-	11
4	Terminal Questions		-	-	12
5	Answers to Self Assessment Questions		-	-	12
6	Answers to Terminal Questions		-	-	12
7	Exercises		-	-	12

1. INTRODUCTION

1.1. Overview in C

C is a general-purpose, structured programming language. Its instructions consist of terms that resemble algebraic expressions, augmented by certain English keywords such as if, else, for, do and while.

1.2. History and evolution of the C programming

C was the offspring of the 'Basic Combined Programming Language' (BPCL) called B, developed in the 1960's at Cambridge University. B language was modified by Dennis Ritchie and was implemented at Bell laboratories in 1972. The new language was named C. Since it was developed along with the UNIX operating system, it is strongly associated with UNIX. This operating system, which was also developed at Bell laboratories, was coded almost entirely in C. In C, the type of a variable determines what kinds of values it may take on. The type of an object determines the set of values it can have and what operations can be performed on it. This is a fairly formal, mathematical definition of what a type is, but it is traditional (and meaningful). There are several implications to remember:

1. The "set of values'' is finite. C's int type cannot represent all of the integers; its float type cannot represent all floating-point numbers.
2. When you're using an object (that is, a variable) of some type, you may have to remember what values it can take on and what operations you can perform on it. For example, there are several operators which play with the binary (bit-level) representation of integers, but these operators are not meaningful for and may not be applied to floating-point operands.
3. When declaring a new variable and picking a type for it, you have to keep in mind the values and operations you'll be needing.

This unit will introduce you to C programming with its various features, structure and how the C programs are constructed with simple examples.

1.3. Objectives:

At studying this unit, you should be able to:

- *Discuss the features of C programming language*
- *Explain the basic structure of a C program*

2. FEATURES OF C AND ITS BASIC STRUCTURE

C is characterized by the ability to write very concise source programs, due in part to the large number of operators included within the language.

It has a relatively small instruction set, though actual implementations include extensive library functions which enhance the basic instructions.

The language encourages users to write additional library functions of their own. Thus, the features and capabilities of the language can easily be extended by the user.

C compilers are commonly available for computers of all sizes. The compilers are usually compact, and they generate object programs that are small and highly efficient when compared with programs compiled from other high-level languages.

Another important characteristic of C is that its programs are highly portable, even more so than with other high-level languages. The reason for this is that C relegates most computer dependent features to its library functions. Thus, every version of C is accompanied by its own set of library functions, which are written for the particular characteristics of the host computer.

A C program can be viewed as a group of building blocks called functions. A function is a subroutine that may include one or more statements designed to perform a specific task. To write a C program we first create functions and then put them together. A C program may contain one or more sections shown in Fig. 1.1.

The documentation section consists of a set of comment (remarks) lines giving the name of the program, the author and other details which the programmer would like to use later. Comments may appear anywhere within a program, as long as they are placed within the delimiters /* and */ (e.g., /*this is a comment*/). Such comments are helpful in identifying the program's principal features or in explaining the underlying logic of various program features.

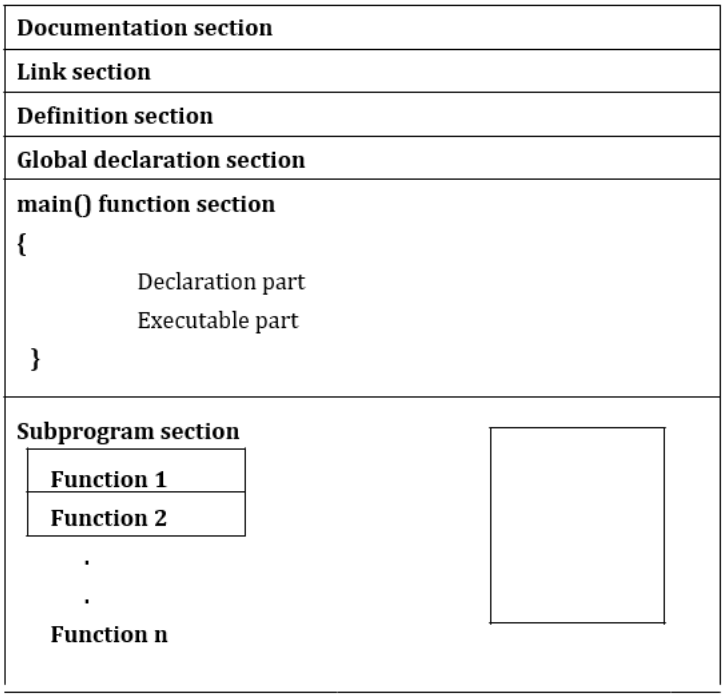

Figure 1.1: Sections in a C program

The link section provides instructions to the compiler to link functions from the system library. The definition section defines all symbolic constants.

There are some variables that are used in more than one function. Such variables are called *global* variables and are declared in the global declaration section that is outside of all the functions.

Every C program must have one **main** function section. This section contains two parts, declaration part and executable part. The declaration part declares all the variables used in the executable part. There is at least one statement in the executable part. These two parts must appear between opening and closing braces ({ and }). The program execution begins at the opening brace and ends at the closing brace. The closing brace of the main function section is the logical end of the program. All statements in the declaration and executable parts end with a semicolon(;).

The subprogram section contains all the user-defined functions that are called in the main function. User-defined functions are generally placed immediately after the **main** function, although they may appear in any order.

All sections, except the **main** function section may be absent when they are not required.

2.1. Programming Style:

Programming style in C involves writing code in a readable and consistent manner. This includes choosing meaningful variable and function names, using indentation for clarity, and following a consistent coding convention. Adopting a good programming style enhances code maintainability and makes it easier for others to understand and collaborate on the codebase.

SELF-ASSESSMENT QUESTIONS - 1

1. Using C language programmers can write their own library functions. (True/False)
2. C is a _____ level programming language.
3. The documentation section contains a set of _____ lines.
4. Every C program must have one **main**() function. (True/False)

2.2. Simple C Programs

Let us see a simple c program given below:

```
#include <stdio.h>
main()
{
printf("Hello, world!\n");
return 0;
}
```

2.3. Executing a C Program:

To execute a C program, it undergoes a compilation process that translates the human-readable source code into machine-readable object code. The resulting executable file can then be run on a computer. Various compilers, such as GCC (GNU Compiler Collection), are commonly used to compile C programs.

If you have a C compiler, the first thing to do is figure out how to type this program in and compile it and run it and see where its output went.

The first line is practically boilerplate; it will appear in almost all programs we write. It asks that some definitions having to do with the "Standard I/O Library'' be included in our program; these definitions are needed if we are to call the library function **printf** correctly.

The second line says that we are defining a function named **main**. Most of the time, we can name our functions anything we want, but the function name **main** is special: it is the function that will be "called'' first when our program starts running. The empty pair of parentheses indicates that our main function accepts no *arguments*, that is, there isn't any information which needs to be passed in when the function is called.

The braces { and } surround a list of statements in C. Here, they surround the list of statements making up the function **main**.

The line **printf("Hello, world!\n");** is the first statement in the program. It asks that the function **printf** be called; **printf** is a library function which prints formatted output. The parentheses surround **printf's** argument list: the information which is handed to it which it should act on. The semicolon at the end of the line terminates the statement.

printf 's first (and, in this case, only) argument is the string which it should print. The string, enclosed in double quotes (""), consists of the words "Hello, world!'' followed by a special sequence: \n. In strings, any two-character sequence beginning with the backslash \ represents a single special character. The sequence \n represents the " 'new line'' character, which prints a carriage return or line feed or whatever it takes to end one line of output and

move down to the next. (This program only prints one line of output, but it's still important to terminate it.)

The second line in the **main** function is

return 0;

In general, a function may return a value to its caller, and **main** is no exception. When **main** returns (that is, reaches its end and stops functioning), the program is at its end, and the return value from **main** tells the operating system (or whatever invoked the program that **main** is the main function of) whether it succeeded or not. By convention, a return value of 0 indicates success.

Program: Area of a circle

Here is an elementary C program that reads in the radius of a circle, calculates the area and then writes the calculated result.

```
#include <stdio.h>                          /* Library file access */
/* program to calculate the area of a circle
*/                                          /* Title (Comment) */
main()                                      /* Function heading */


float radius, area;                         /*Variable declarations */
printf("Radius=?");                         /* Output statement(prompt) */
scanf("%f", &radius);                       /* Input statement */
area=3.14159*radius*radius;                 /* Assignment statement */
printf("Area=%f",area);                     /* Output statement */
```

Program: Print a few numbers

Here is a program to illustrate a simple loop

```
#include <stdio.h>
/* print a few numbers, to illustrate a simple loop */
main()
{
int i;
for(i = 0; i < 10; i = i + 1) /* Looping statement */ printf("i is %d\n", i);

return 0;
}
```

Program: Program to add two numbers

```
#include <stdio.h>
main()
{
int i,j,k; // Defining variables
i = 6;    // Assign values
j = 8;
k = i + j;
printf("sum of two numbers is %d \n",k); // Printing results
}
```

SELF-ASSESSMENT QUESTIONS - 2

5. The information that needs to be passed in when a function is called is_____.
6. The main() function doesn't return any value. (True/False)

3. SUMMARY

C is a pivotal programming language with a rich history and evolution that traces back to its development in the early 1970s by Dennis Ritchie at Bell Labs. Serving as the successor to the B programming language, C became integral to the creation of the Unix operating system and influenced the development of numerous other programming languages. The standardization of C occurred with the establishment of the ANSI C standard in 1989 and the subsequent ISO C standard in 1999.

The basic structure of a C program is characterized by a preprocessor directive, function declarations, the main function, and additional user-defined functions. Programs are organized into modular functions, each tasked with specific responsibilities. Syntax rules dictate declarations and statements within functions.

Programming style in C emphasizes readability and consistency. Adopting meaningful variable and function names, using indentation, and adhering to coding conventions contribute to a more maintainable and understandable codebase. A well-defined programming style facilitates collaboration and code comprehension.

To execute a C program, a compilation process is employed. This process translates human-readable source code into machine-readable object code, resulting in an executable file. Popular compilers like GCC play a crucial role in compiling C programs. Overall, C programming offers a robust foundation for software development, and understanding its basic structure, adhering to programming style guidelines, and navigating the compilation and execution process are essential aspects of effective C programming.

4. TERMINAL QUESTIONS

1. Explain the history of C language.
2. What are the advantages of C language?
3. What are the major components of a C program?
4. What significance is attached to the function main?

5. ANSWERS TO SELF ASSESSMENT QUESTIONS

1. True
2. High
3. Comment
4. True
5. Arguments
6. False

6. ANSWERS TO TERMINAL QUESTIONS

1. C was developed by Dennis Ritchie and came after B in the year 1972. (Refer Section 1.1 for more details)
2. C is fast, efficient and structured. (Refer to section 1.2 for more details)
3. Documentation section, Link section, Definition section, Global declaration section, main() function section, Subprogram section
4. main() is the function that will be “called'' first when our program starts running.

7. EXERCISES

1. Explain the basic structure of a C program with an example.
2. What are the different steps in executing a C program?
3. Write a C program to convert Celsius to Fahrenheit and vice versa.
4. Write a program in C to add, subtract, multiply any 3 numbers.